

A Preliminary Roadmap for Empirical Research on Agile Software Development

Torgeir Dingsøyr¹, Tore Dybå¹, Pekka Abrahamsson^{1,2}

¹SINTEF ICT, NO-7465 Trondheim, Norway

²VTT Technical Research Centre of Finland, P.O. Box 1100, FIN-90571 Oulu, Finland

[torgeir.dingsoyr|tore.dyba|pekka.abrahamsson}@sintef.no](mailto:{torgeir.dingsoyr|tore.dyba|pekka.abrahamsson}@sintef.no)

Abstract

Some claim that especially in the field of agile software development the research lags years behind of the practice. In this paper, we characterize the status and main challenges for research on agile software development, and propose a preliminary roadmap, focusing on providing more empirical research, primarily on experienced agile teams and organizations, connecting better to existing streams of research in more established fields, giving more attention to management-oriented approaches, and finally give more emphasis to the core ideas in agile software development in order to increase our understanding. We hope that this preliminary roadmap serves as a starting point for creating a common research agenda and enables the generation of fruitful discussions and research results from the field.

1. Introduction

The ideas which compose agile software development have had a major impact on how software engineering is done in practice the past years. The 1990 summer issue of American Programmer was devoted to ‘Peopleware’, and the editor commented that “Everyone knows the best way to improve software productivity and quality is to focus on people.” This is exactly what has been done in many of the agile development methods suggested by practitioners, such as Extreme Programming [7], Scrum [26], Feature-Driven Development [21], and Lean Software Development [23]. Ågerfalk and Fitzgerald [35] have argued that “practice is ahead of research” in this area. As an example, a recent industrial report cites more than 60 industrial trials of the application of agile software development methods in the area of embedded system development involving more the 2,000

engineers in 17 European companies [1]. Rajlich further claims that agile software development brings a host of new topics into software engineering research, that there exists a “backlog of research tasks” [24].

As we will describe in section two, research on agile development is characterized by a small number of empirical research papers, on a wide range of topics. This makes it difficult for research to provide advice for industry grounded in research findings. The lack of studies on important topics also is a barrier towards our understanding of agile software development. In the following, we will show that the current situation may even be risky as the agile development approaches are keenly adopted by industry, while the number of empirical research papers increases only slowly. This is a barrier to bridging the gap between research and practice. Therefore, in order to improve this situation, we suggest that researchers and practitioners in the field should collaborate to determine what topics should be given priority in the “backlog”, to provide a common research agenda or a roadmap. Examples of such roadmaps in other fields can be found at the Future of Software Engineering track at the International Conference on Software Engineering [9]. These roadmaps serve as higher level pointers for research in software engineering in general, and require adaptation with regard to different software engineering topics. The purpose of this paper is to propose a preliminary roadmap for research on agile software development.

The paper is structured as follows: We first give a brief overview of the current state of research in related core fields: Software engineering, information systems, and empirical software engineering. Then, we summarize existing overview studies on agile software development, and emphasize publication channels, research methods applied, and the topic areas. We then proceed to outline a preliminary

roadmap, which we hope can lead to a discussion on what the main important areas for research are for the next five to ten years. Finally, we outline what we see as the main tasks for the field to meet the challenges, and conclude.

2. Background: Current state of research

Agile software development is a phenomenon which has mainly been studied in the fields of software engineering and information systems. As such, the challenges of these fields also apply to research on agile software development. In the analysis of research in the computing disciplines, Glass *et al.* [18] finds that software engineering focuses primarily on topics related to systems and software, like methods and techniques for software development and the software life cycle, while information systems research is more focused on organizational concepts such as information technology usage and technology transfer. The information systems field applies theories from a number of other fields, such as management, economics and cognitive psychology, while 98% of the references in software engineering are within the same field.

A general challenge identified in software engineering has been to increase the number of empirical studies, and depend less on proposed methods and techniques supported by arguments alone. This bears direct impact on practice. While software professionals seek a rational basis for making a decision about which methods they should adopt, the basis for such a rationalization is often missing. Methods introduced continue to be based more on faith than on an empirical data [17].

Selby *et al.* [27] summarize the discussions from the second Dagstuhl seminar on empirical software engineering, and suggest a roadmap for research where current status and goals for the future is described in four main areas: 1) The maturity of the field, in terms of cohesiveness of research, and the research methodology. 2) The coverage of research, in terms of which processes, techniques or phases, problem domains, artefact scales and expertise levels have been covered. 3) Understanding, what kind of evidence we have for proposed theories, and finally: 4) Impact, what kind of impact the empirical studies have on fields related to software engineering, and what kind of impact the research has on the software industry.

The focus on empirical research in software engineering was addressed at the Future of Software Engineering track at ICSE 2007: Sjöberg *et al.* [31]

identify the major challenges for the research field, which include increasing the number of empirical research studies, a better synthesis of empirical evidence and a larger number of theories built and tested in the field. Common research agendas are seen as a measure in order to reach such goals.

2.1 Overviews of agile research

Introductions to and overviews of agile development are given by Abrahamsson *et al.* [3], Cohen *et al.* [11], Erickson *et al.* [16] and Dybå and Dingsøyr [13]. These four reports describe the state of the art and state of the practice in terms of characteristics of the various agile methods, lessons learned and empirical studies from applying such methods in industry. We summarize each of these four overviews briefly:

The first review of the existing literature on agile software development was done in a technical report published by Abrahamsson *et al.* in 2002 [3]. The report discusses the concept of agile development, presents processes, roles, practices, and experience with ten agile development methods, and compares the methods with respect to the phases that they support and the level of competence that they require. Only DSDM and the Rational Unified Process were found to give full coverage to all phases of development, while Scrum mainly covers aspects related to project management. Abrahamsson *et al.* found anecdotal evidence that agile methods are “effective and suitable for many situations and environments”, but state that very few empirically validated studies support these claims. The report was followed by a comparative analysis of nine agile methods in 2003 [4], where it is stated that empirical support for the suggested methods remains scarce.

Cohen *et al.*'s review published in 2004 [11] emphasizes the history of agile development, shows some of the roots to other disciplines, and, in particular, discusses relations between agile development and the Capability Maturity Model (CMM). They further describe the state of the art with respect to the main agile methods and their characteristics. They discuss issues such as the introduction of, and project management in, agile development. They present experiments and surveys, and seven case studies of agile development. The authors believe that agile methods will be consolidated in the future, just as object-oriented methods were consolidated. Further, they do not believe that agile methods will rule out traditional methods. Rather, they believe that agile and traditional methods will have a symbiotic relationship

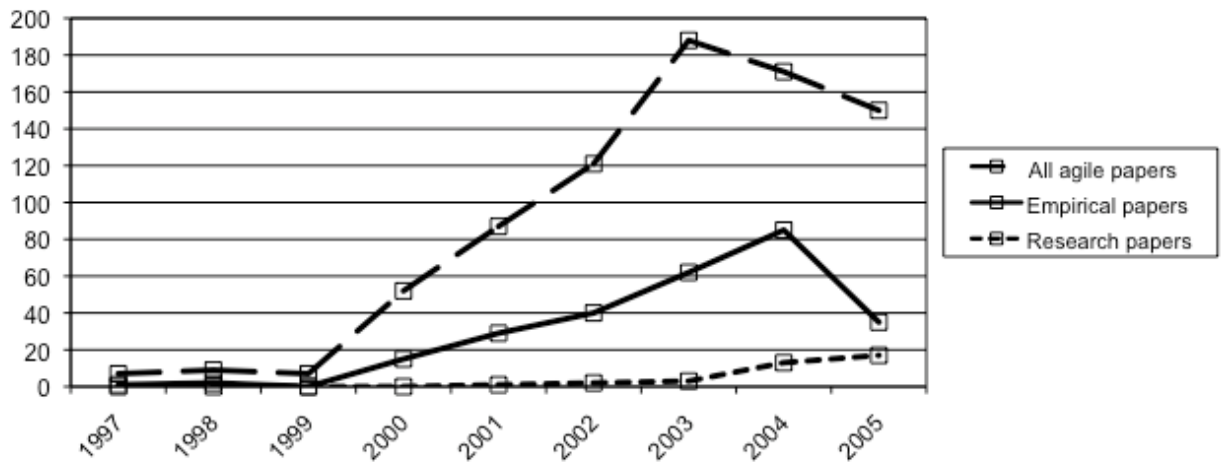


Figure 1: Number of papers identified in systematic review by year: All agile papers, empirical papers and empirical research papers.

in which factors such as the number of people working on a project, application domain, criticality, and innovativeness will determine which process to select.

In 2005, Erickson et al. [16] described the state of research on XP, agile software development, and agile modelling. With respect to XP, they found a small number of case studies and experience reports that promote the success of XP. They found that the XP practice of pair programming is supported by a more well-established stream of research, and that there are some studies on iterative development. Erickson *et al.* recommend that the other core practices in XP be studied separately in order to identify what practices are working. Furthermore, they see challenges with matching agile software development methods with standards such as ISO, and they argue that this is an area that needs further research. They found much less research on agile modelling than on XP.

Dybå and Dingsøyr's [13] systematic review identified 33 primary empirical studies on agile software development, which report observational or other data about how agile development is conducted. The studies fell into four thematic groups: introduction and adoption, human and social factors, perceptions of agile methods, and comparative studies. The study identified a number of reported benefits and limitations of agile development within each of these themes. The studies investigated XP almost exclusively, but only a few of the studies on XP were done on experienced agile software development teams. A clear finding of the review is

that we need to increase both the number and the quality of studies on agile software development. In particular, agile project management methods, such as Scrum, which are popular in industry, warrant further attention.

Regarding the year of publication (see Figure 1), Dybå and Dingsøyr [13] found a growing number of papers published on agile software development until 2003 (all agile papers). About one third of the agile papers contained quantitative or qualitative material, including lessons learned reports (empirical papers). Of these, there were 33 primary empirical research studies (research papers). All in the latter category were published after 2000. From 2001, Dybå and Dingsøyr found a steady increase of studies with one empirical research study published in 2001, one in 2002, three in 2003, 12 in 2004, and 16 published in 2005.

2.2. Channels and methods for agile research

When discussing a roadmap we find it important to understand which arenas have been concerned with topics related to agile software development. In particular the distribution between journal and conference publications is an indicator of maturity of the field and of subfields. Table 1 from Dybå and Dingsøyr [13] gives an overview of the studies according to publication channel. We see that the conferences XP and Agile Development have the largest number of studies. Most of the studies were

Table 1: Distribution of studies according to publication channel.

Publication Channel	Type	Number	Percent
XP 200X	Conference	5	15
Agile Development Conference	Conference	5	15
IEEE Software	Journal	3	9
Profes	Conference	2	6
ASEE FEC	Conference	1	3
Computer Supported Cooperative Work	Journal	1	3
CSMR	Conference	1	3
Empirical Software Engineering	Journal	1	3
ENCBS	Conference	1	3
ENC	Conference	1	3
EuroMicro	Conference	1	3
EuroSPI	Conference	1	3
HICCS	Conference	1	3
HSSE	Conference	1	3
IASTED ICSEA	Conference	1	3
ICSE	Conference	1	3
ISESE	Conference	1	3
ITIICT	Conference	1	3
Journal of database management	Journal	1	3
Metrics	Conference	1	3
Software Quality Journal	Journal	1	3
XP/Agile Universe	Conference	1	3
Total		33	100

published in conferences, 79%, while 21% appeared in scientific journals. Note that 3 of the 7 articles categorized as scientific journal articles were published in IEEE Software, which is a magazine, and not a primary academic software engineering journal.

The number and percentage of publications using each research method in the study by Dybå and Dingsøyr [13] is listed in Table 2. Of the 13 single-case studies, nine were done in projects in industry. The material for the other four studies was taken

from projects where students did the development. Interestingly, three of these studies took their data from the same project. Only one of the single-case studies in industry was done with a team who had prior experience with agile methods.

For the multiple-case studies (11 in total), all were done in industry, but only three of the studies were on software development teams with prior experience with agile methods. The number of cases varied from two to three. Three of the four surveys were done on employees in software companies, while one was done on students. The three experiments were all done on students, with team sizes ranging from three to 16.

Table 2: Research method

Research method	Number	Percent
Single-case	13	39
Multiple-case	11	33
Survey	4	12
Experiment	3	9
Mixed	2	6
Total	33	100

2.3. Focus areas for agile research

With respect to the kinds of agile method that have been studied, we see from Table 3 that 76% of the studies in the review by Dybå and Dingsøyr [13] were done on XP. Studies on agility in general come next, with 15% of the studies. Scrum and Lean Software Development were studied in only one empirical research article each.

Table 3: Studies after type of agile method used in the study.

Agile method	Number	Percent
XP	25	76
General ¹	5	15
Scrum	1	3
Lean Software Development	1	3
Other ²	1	3
Total	33	100

¹“General” refers to studies on agility in general

²“Other”: refers to a company-internal agile method.

The review reports that 73% of the studies that investigated agile projects dealt with employees who are beginners (less than a year of experience in agile development). Twelve per cent of the studies dealt with mature agile development teams (at least one year of experience in agile development). Two studies did not indicate whether it was a beginner or mature team that was studied and for three studies (surveys) this classification was not applicable.

Most studies, 73%, dealt with professional software developers. The remaining 27% were conducted in a university setting. Most projects were of short duration and were completed by small teams.

An unpublished study of topics related to agile software development in conference proceedings found 151 papers, 90 of which claimed that their findings were based on empirical evidence [2]. The conferences included XP, Agile, Agile-Universe, ICSE and Profes. This search differs from the literature search in the systematic review by Dybå and Dingsøyr [13] in that it not only includes empirical studies on whole agile methods, but also studies on techniques. Also, the criteria for what was considered an empirical study differed.

The 90 papers were divided in several subcategories based on their primary area of interest (Figure 2). It was found that a great majority (42%) of the publications focus on XP and more specifically on its practices, confirming the results of Dybå and Dingsøyr’s systematic review. The remaining studies were spread to a large number of different topics: testing (7%), large & complex systems (7%), education (6%), Scrum (6%) and quality (6%). Other areas included human values and satisfaction, requirements, tailoring and collaboration & communication each having less than 5% of papers. The second largest group, however, was the “other” focus (15%) not belonging to any subgroups already mentioned. There are therefore a number of individual papers addressing e.g. research methods,

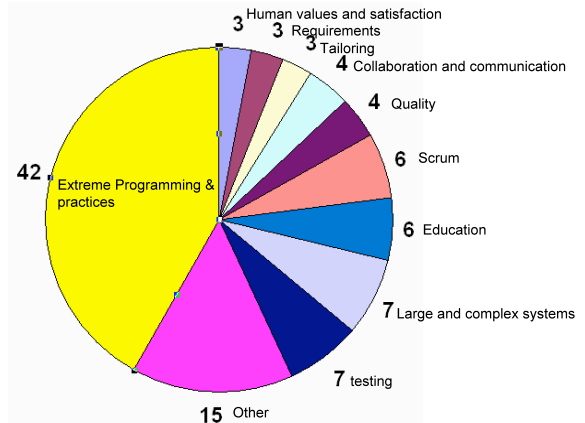


Figure 2: Distribution of conference papers according to topic.

documentation, outsourcing and reusability aspects of agile software development.

One of the main streams of agile research relates to pair programming, one of the core practices in XP. Advocates of pair programming claim that it has many benefits over individual programming when applied to new code development or when used to maintain and enhance existing code. Claimed benefits include higher-quality code, happier programmers, improved teamwork, improved knowledge transfer, and enhanced learning [34]. There are also claims with respect to the benefits and drawbacks of various kinds of pairing, e.g., that “expert-expert pairings seem to be especially accelerated” (ibid.) and that “novice-novice pairing is akin to the blind leading the blind” [33].

A meta-analysis of pair-programming [12] found a general agreement among the 15 included studies that pair programming leads to increased quality and shorter development time, but that this comes at a cost of greater effort. The largest study included in the meta-analysis, was the experiment conducted by Arisholm *et al.* [5], in which 295 Java consultants was hired to examine how effectively participants at three levels of expertise (junior, intermediate, and senior) made changes to systems with two distinct designs (one easier than the other). The results showed that by cooperating, programmers could complete tasks and attain goals that would be difficult or impossible if they worked individually. Yet, the higher quality for complex tasks came at a price of a considerably higher effort (cost), while the reduced completion time for the simpler tasks came at a price of a noticeably lower quality. In particular, the experiment showed that junior pair programmers achieved a significant increase in quality compared with the individuals and achieved approximately the same degree of quality as senior individuals.

A second stream of research deals with test-first programming [15] or test-driven development. This is one of the most controversial areas of study in agile software development. While some claim that the practice dates back to 1970's or beyond [20], it fundamentally changes the way software is being developed. The thinking changes from "how it should be developed?" to "how it should be tested?" The latter guides the development process. Proponents of the approach claim that several positive benefits can be achieved by the application of the practice. Siniaalto and Abrahamsson [30] reviewed the existing literature on test-driven development (up to and including 2006) and found that there are 16 empirical studies on the topic. Seven of these studies were purely academic endeavours with student subjects performing an experimental task or a set of tasks. Four of the studies were performed in industry with experienced developers. Five of the remaining studies, according to the authors, were semi-industrial. They explained the semi-industrial setting composing of either experienced developers and an experimental task or student developers and a real system. The authors summarize the findings and state that test-first programming may improve software quality significantly. They also state that productivity improvements are less obvious and that very little evidence exists on the impact of test-first programming on program design, which is one of the most fundamental arguments that the test-first proponents have laid out.

A third major stream of research related to extreme programming focuses on the relevance of using XP in education. Many reports can be found in the literature on how XP has been used in university courses, for example at the North Carolina State University [29] and, University of Sheffield [19]. Schneider and Johnston [26], however, argue that XP has limited value with respect to meeting educational goals in undergraduate university courses, because limited resources at the universities makes it difficult to set up a necessary infrastructure, and the code-centric view of XP makes it difficult to introduce

practices needed for larger-scale systems. Finally, they claim that the university students do generally not have the maturity to understand the practices and their consequences. The suitability of agile methods in education is thus an ongoing debate.

3. A preliminary roadmap

We structure the proposal for a preliminary roadmap for agile research in the template used by Selby *et al.* [28] in the Dagstuhl seminar on empirical software engineering, presenting what we view as the status and a good goal for agile research for 2015, see Table 4. Figure 3 sums up our assessment of the current status, goal for 2015 and general goals for empirical software engineering. We suggest that agile research should develop on all axes, but argue that some directions should be given priority. In the following, we present status and goals when it comes to maturity, coverage, understanding and impact. See the appendix for progress indicators for dimensions in these four categories.

3.1. Maturity of research

Edmondson and McManus [14] divide the current state of theory and research into three categories: nascent, intermediate, and mature. Nascent research areas are characterized by open-ended inquiry about a phenomenon, which can generate suggestive theories. The intermediate areas have proposed relationships between new and established constructs and can provide provisional relationships between new and established constructs. Finally, a mature area is characterized by focused research questions or hypotheses relating existing constructs, and lead to new or more precise theory which is well supported.

From the previous section, we see that the current state of theory and research on particular agile development methods is nascent, while some areas might be characterized as intermediate, see Table 4.

Table 4: Categorization of agile topics with respect to state of theory and research.

Nascent	Intermediate	Mature
Agile modelling DSDM Lean Software Development Scrum Standardization and agile software development XP in software engineering education	Extreme programming (XP) XP technique pair programming XP technique test-driven development	

Table 5: Description of current status and suggested goal for 2015.

Roadmap Area	Description	Status 2008	Goal 2015
Maturity	Cohesiveness of the field Research methodology	Some joint large research efforts, however mainly in Europe Methodological problems as in software engineering Initial standardization work under way	Several large research projects. Research methodology as mature as in information systems.
Coverage	Process/technique/phase Problem domain Artefact scale Subject expertise level	XP and some techniques covered for inexperienced teams	Significantly increased coverage of agile software development methods for experienced software development teams and large organizations. Good coverage of management-oriented approaches.
Understanding	Evidence	Intermediate level in certain areas, nascent on most	Mature research on certain topic areas, intermediate on a number of areas. Significantly better understanding on what agile software development is.
Impact	Industry Science/Engineering Education	Little impact on how agile development is carried out in industry, and little impact on other research fields, some impact in education	Research has a significant impact on certain areas of agile software development on industry, science and education.

The grand challenge for agile research is to combine research rigour with industrial relevance [8, 25]. We view research methodology as an area where there is room for substantial improvement in agile research.

Research rigour should not only be understood as having proper mechanisms for research design, data collection and analysis, but also ensure that the research is contributing to a better understanding of the topic area. Here, we propose introducing relevant theories from other disciplines and building theories bottom-up can give a major contribution. Introducing relevant theories not only applies to related fields like software engineering and lean production. Many theories in use in the information systems field are highly relevant also for research on software development in general and in agile software development in particular. The web resource¹ devoted to the theories in use in information systems lists theories such as task-technology fit, theory of planned behaviour, contingency theory, complexity theory, social learning theory, social network theory, socio-technical theory, organizational learning theory, and the knowledge-based theory of the firm, which all are used in information systems and seem relevant to agile development. Also theories from

other domains, such as business theories should also be relevant, as agile methods impact beyond immediate software development settings. Such a focus should lead to more articles on agile software development appearing in scientific journals.

For building theories, Carrol and Swatman [10] define three levels of sophistication for information systems:

1. Minor working relationships that are concrete and based directly on observations.
2. Theories in the middle range that involve some abstraction but are still closely linked to observations
3. All-embracing theories that seek to explain social behaviour.

Sjøberg *et al.* [32] discuss theory building in software engineering, and give an example of going from an exploratory case study to theory using grounded theory. This is in our opinion very relevant also for research on agile software development, and we should aim for theories in the middle range as defined by Carrol and Swatman.

Another indication of maturity of a field is the cohesiveness of research. For agile development, there have been some larger industry-driven research

¹ http://www.fsc.yorku.ca/york/istheory/wiki/index.php/Main_Page

projects in Europe, such as AGILE-ITEA and FLEXI-ITEA2, and there are conferences like Agile and XP which contribute to more cohesive research, but we believe most research is driven by individual research plans. We therefore argue that this is an area that needs to be given priority, and a discussion of this roadmap is a contribution in this direction.

3.2. Coverage

When it comes to coverage of the research area, Selby *et al.* [28] divided this in several subareas: coverage of processes, techniques and phases (single technique to comprehensive processes), coverage of the problem domain (single artefact to comprehensive coverage), coverage of artefact scales (from units to systems of systems) and coverage of subject expertise level (junior to advanced).

As for coverage of processes, techniques, and phases, we saw in section 2 that it is primarily XP that is covered today. For agile methods, we think the priority should be given to studying methods that have gained industrial acceptance but have been given little attention in research. Management-oriented approaches such as Scrum is an example of an area where we think there is a large gap, and should be given priority.

When it comes to artefacts covered, we see that research on XP techniques pair programming and test first development are examples of single techniques in an intermediate stage. Also, the whole process of XP is covered at this level.

We maintain that coverage of subject expertise level is an area that should be given priority. When studying agile development practices in vivo, we argue that more studies should be carried out on experienced agile software development teams and organizations. If we want to know what agile methods can achieve, this is the most interesting target group.

Another issue is that there is a need for more longitudinal studies in order to examine how effects of agile development sustain over time, and how they affect other parts of the organization such as sales, business management and other stakeholders.

A larger body of primary studies will also give us more knowledge on how agile principles work in different contexts, for example in safety critical systems development, hardware-bound software development or in organizations offering software

services, in order to find how to apply agile principles in different real-life situations.

An issue related to increasing the problem domain is to understand the complicated dynamics of agility, what are the core characteristics of agile software development, and what is similar and different from agility in other disciplines? Are there underlying principles that have to be in place in agile development which we are not aware of? How do the various practices and recommendations in agile development relate to established fields on issues like project management, requirements engineering, customer collaboration, team effectiveness, knowledge management, method engineering and organizing the architecture of the software?

3.3. Understanding

The understanding of a field is described from having evidence of claims from research conjectures to replicated results and validated theories. From the systematic review on agile software development [13] and the description of nascent and intermediate topic areas in agile software development, there are supportive observations on some themes. These themes include the development of micro level understanding on specific techniques such as pair programming and test-driven development. However, we still need more primary studies of high quality in order to see common patterns in different contexts.

3.4. Impact

The roadmap framework distinguished between two types of impact: Impact on the software engineering industry, and impact on other fields of science. The industrial relevance of agile software development is clear. However, one could argue that research has had little impact on everyday practice in software organizations. Increased application of research methods like action research [6] can be helpful in ensuring the relevance, and help provide a larger body of knowledge which can lead to a broader impact on industry.

Regarding impact on science, agile software development has had some impact on software engineering education, but little on other research fields.

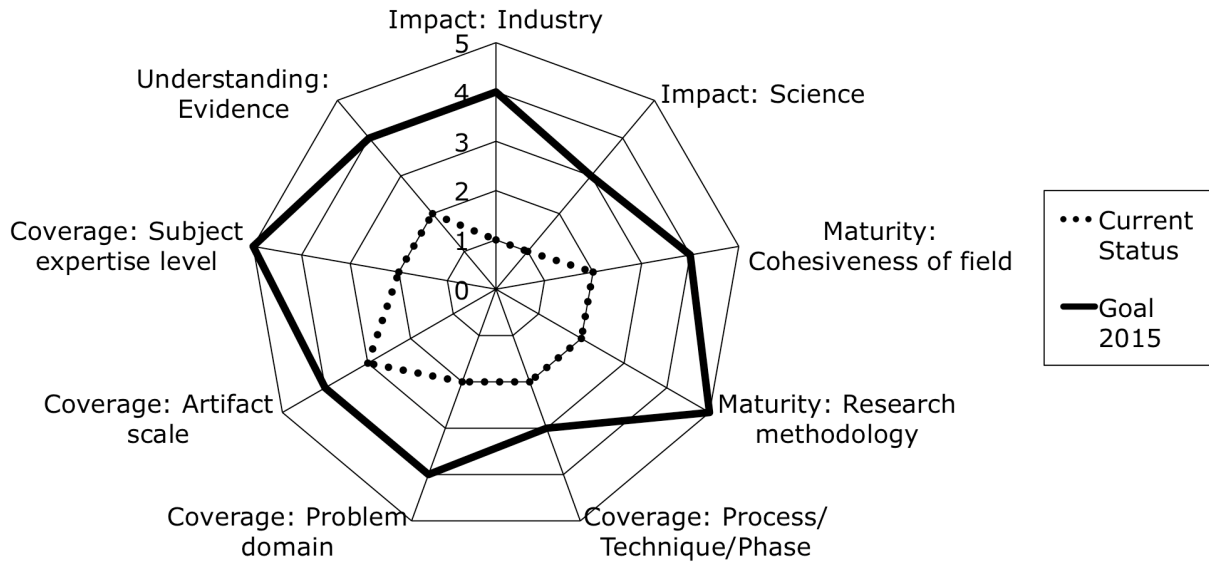


Figure 3: Preliminary roadmap, showing our assessment of the current status for agile software development and a suggested goal for 2015.

4. How to meet the challenges?

Table 5 and Figure 3 summarize in words and scales what we believe the current status is in research on agile software development. We argue in line with others [i.e., 31] that there is an urgent need to increase the body of empirical studies in software engineering in general. Measures discussed to achieve this has included integrating courses on empirical methods in software engineering education, providing guidelines for applying various empirical methods, increasing the collaboration between academia and industry, increasing the resources available for empirical research, and finally establishing common research agendas.

We believe all these efforts are relevant for agile software development. However, for the research themes to be addressed, this requires a discussion which has to be taken within the agile research community.

In the previous section we have argued that we need particular focus on the following areas:

Maturity: Increase the cohesiveness of the field through discussions on what directions should be given priority.

Improve the research methodology applied in studies in agile software development. This is a challenge we share with the field of software engineering. A particular issue of importance here is to connecting the empirical studies of agile development with existing streams in more established fields.

Coverage: Increase the coverage of subject expertise level, mainly by studying experienced agile development teams and organizations. Focus should also be placed on management-oriented approaches such as Scrum, as this has both impact and potential in industry, but has received very little attention from the research community. For example, studies in sociotechnical systems suggest that self-organization, which is the guiding principle in Scrum, is not necessarily suitable in complex and non-linear problem-solving situations [22]. These types of situations are likely to be found in large systems development.

A larger base of primary studies will enable us to develop theory from empirical studies. For example, special issues in journals is a mechanism to stimulate a larger number of primary studies. We also think a higher emphasis on core ideas in agile software development would be very useful in order to increase our understanding

Impact: Increase the impact of research on practice and other fields by providing more solid research, in particular on the topics mentioned above.

We hope these key points can help to make research more relevant for industry, and also in providing a better understanding of agile software development in complex real-life development settings.

5. Conclusion

We have given an overview of the current state of the art in research on agile software development, identified research channels and methods applied, and identified main topics of research. Using the framework of Edmondson and McManus [14], we characterized the topic areas on agile software development as nascent, intermediate and mature, and we used the framework by Selby *et al.* [28] to describe the current status of research and goals for the field in 2015. From this we suggest a preliminary roadmap for research on agile software development, and suggest that researchers in the field should increasingly address the challenges by focusing on providing more empirical research, primarily on experienced agile teams and organizations, connecting better to existing streams of research in more established fields, giving more attention to management-oriented approaches, and finally give more emphasis to core ideas in agile software development in order to increase our understanding.

We hope that this preliminary roadmap serves as a starting point for creating a common research agenda and enables the generation of more fruitful research results from the field.

Acknowledgement

The work in this paper was supported by the Research Council of Norway through the project Evidence-Based Software Engineering (181685/I30), National Technology Agency of Finland, and the FLEXI-ITEA2 research project. We would also like to thank the anonymous reviewers to for helpful comments.

References

- [1] Abrahamsson, P., “Speeding up Embedded Software Development: Application of Agile Processes in Complex Systems Development Projects,” ITEA Innovation Report, Information Technology for European Advancement, ITEA2 2007, http://www.itea-office.org/innovation_reports
- [2] Abrahamsson, P., “What Do We Empirically Know About Agile Software Development,” lecture held at Information Technology Postgraduate Training Programme (INFWEST.IT), Oulu, Finland, April, 2007.
- [3] Abrahamsson, P., Salo, O., Ronkainen, J., and Warsta, J., “Agile Software Development Methods: Review and Analysis,” VTT Publications 478, Espoo, Finland, 2002.
- [4] Abrahamsson, P., Warsta, J., Siponen, M. T., and Ronkainen, J., “New Directions on Agile Methods: A Comparative Analysis,” in *Proceedings of the 25th International Conference on Software Engineering (Icse'03)*: IEEE Press, 2003.
- [5] Arisholm, E., Gallis, H., Dybå, T., and Sjøberg, D. I. K., “Evaluating Pair Programming with Respect to System Complexity and Programmer Expertise,” *IEEE Transactions on Software Engineering*, no. 2, vol. 33, pp. 65-86, 2007.
- [6] Avison, D., Lau, F., Myers, M., and Nielsen, P. A., “Action Research,” *Communications of the ACM*, no. 1, vol. 42, pp. 94-97, 1999.
- [7] Beck, K., *Extreme Programming Explained: Embrace Change*: Addison-Wesley, 2000, ISBN 201-61641-6.
- [8] Benbasat, I. and Zmud, R. W., “Empirical Research in Information Systems: The Practice of Relevance,” *MIS Quarterly*, no. 1, vol. 23, pp. 3-16, 1999.
- [9] Briand, L. C. and Wolf, A. L., *Future of Software Engineering*: IEEE Computer Society, 2007, ISBN: 978-0-7695-2829-8.
- [10] Carrol, J. and Swatman, P. A., “Structured-Case: A Methodological Framework for Building Theory in Information Systems Research,” *European Journal of Information Systems*, vol. 9, pp. 235-242, 2000.
- [11] Cohen, D., Lindvall, M., and Costa, P., “An Introduction to Agile Methods,” in *Advances in Computers, Advances in Software Engineering*, vol. 62, M. V. Zelkowitz, Ed. Amsterdam: Elsevier, 2004.
- [12] Dybå, T., Arisholm, E., Sjøberg, D., Hannay, J., and Shull, F., “Are Two Heads Better Than One? On the Effectiveness of Pair-Programming,” *IEEE Software*, no. 6, vol. 24, pp. 10-13, 2007.
- [13] Dybå, T. and Dingsøyr, T., “Empirical Studies of Agile Software Development: A Systematic Review,” *Information and Software Technology*, vol. doi: 10.1016/j.infsof.2008.01.006, 2008.
- [14] Edmondson, A. C. and McManus, S. E., “Methodological Fit in Management Field Research,” *Academy of Management Review*, no. 4, vol. 32, pp. 1155-1179, 2007.
- [15] Erdogmus, H., Morisio, M., and Torchiano, M., “On the Effectiveness of the Test-First Approach to Programming,” *IEEE Transactions on Software Engineering*, no. 3, vol. 31, pp. 226-237, 2005.
- [16] Erickson, J., Lyytinen, K., and Siau, K., “Agile Modeling, Agile Software Development, and Extreme Programming: The State of Research,” *Journal of Database Management*, no. 4, vol. 16, pp. 88 - 100, 2005.

- [17] Fenton, N., "Viewpoint Article: Conducting and Presenting Empirical Software Engineering," *Empirical Software Engineering*, vol. 6, pp. 195-200, 2001.
- [18] Glass, R. L., Ramesh, V., and Vessey, I., "An Analysis of Research in Computing Disciplines," *Communications of the ACM*, no. 6 (June 2004), vol. 47, pp. 89 - 94, 2004.
- [19] Kalra, B., Thomson, C., and Holcombe, M., "The Software Hut - a Student Experience of Extreme Programming with Real Commercial Clients," XP, 2005.
- [20] Larman, C. and Basili, V. R., "Iterative and Incremental Development: A Brief History," *IEEE Computer*, no. 6, vol. 36, pp. 47-56, 2003.
- [21] Palmer, S. R. and Felsing, J. M., *A Practical Guide to Feature-Driven Development*. Upper Saddle River, NJ: Prentice Hall, 2002, ISBN 0-13-067615-2.
- [22] Pava, C., Redesigning Sociotechnical Systems Design: Concepts and Methods for the 1990s, *Journal of Applied Behavioural Science* 22 201-221, 1986.
- [23] Poppendieck, M. and Poppendieck, T., *Lean Software Development - an Agile Toolkit for Software Development Managers*. Boston: Addison-Wesley, 2003, ISBN 0-321-15078-3.
- [24] Rajlich, V., "Changing the Paradigm of Software Engineering," *Communications of the ACM*, no. 8, vol. 49, pp. 67 - 70, 2006.
- [25] Robey, D. and Markus, M. L., "Beyond Rigor and Relevance: Producing Consumable Research About Information Systems," *Information Resources Management Journal*, no. 1, vol. 11, pp. 7-15, 1998.
- [26] Schneider, J. G. and Johnston, L., "Extreme Programming – Helpful or Harmful in Educating Undergraduates," *Journal of Systems and Software*, vol. 74, pp. 121-132, 2005.
- [27] Schwaber, K. and Beedle, M., *Agile Software Development with Scrum*. Upper Saddle River: Prentice Hall, 2001,
- [28] Selby, R. W., "Empirical Software Engineering Research Roadmap: Discussion and Summary " in *Empirical Software Engineering Issues. Critical Assessment and Future Directions, Lecture Notes in Computer Science*. Berlin / Heidelberg: Springer Verlag, 2007, pp. 184-187.
- [29] Shukla, A. and Williams, L., "Adapting Extreme Programming for a Core Software Engineering Course," in *Proceedings. 15th Conference on Software Engineering Education and Training (Csee&T)*, 2002, pp. 184-191.
- [30] Siniaalto, M. and Abrahamsson, P., "A Comparative Case Study on the Impact of Test-Driven Development on Program Design and Test Coverage," in *Empirical Software Engineering and Measurement, 2007. Esem 2007. First International Symposium On: IEEE Computer Society*, 2007.
- [31] Sjøberg, D., Dybå, T., and Jørgensen, M., "The Future of Empirical Methods in Software Engineering Research," in *Future of Software Engineering (Fose '07): IEEE*, 2007, pp. 358-378.
- [32] Sjøberg, D. I. K., Dybå, T., Anda, B. C. D., and Hannay, J. E., "Building Theories in Software Engineering," in *Guide to Advanced Empirical Software Engineering*, F. Shull, J. Singer, and D. I. K. Sjøberg, Eds. London: Springer verlag, 2008, pp. 312-336.
- [33] Stephens, M. and Rosenberg, D., *Extreme Programming Refactored: The Case against Xp*. Berkeley, CA: Apress, 2003, ISBN 1-59059-096-1.
- [34] Williams, L. and Kessler, R., *Pair Programming Illuminated*: Addison-Wesley, 2002,
- [35] Ågerfalk, P. and Fitzgerald, B., "Flexible and Distributed Software Processes: Old Petunias in New Bowls?," *Communications of the ACM*, no. 10, vol. 49, pp. 27-34, 2006.

Appendix: Roadmap progress indicators

The following roadmap progress indicators were developed for use in the empirical software engineering research roadmap [27]:

- Category: Maturity
 - *Dimension: Cohesiveness of field*
 - Individual research plan
 - Common terminology
 - Guidelines
 - Standards
 - Common research plan
 - Handbook
 - *Dimension: Research methodology*
 - Subjective views
 - Isolated techniques
 - Understand technique tradeoffs
 - Integrated techniques
 - Repeatable methods
 - Objective framework, standards
- Category: Coverage
 - *Dimension: Process/technique/phase*
 - Single technique/phase
 - Multiple techniques or phases
 - Multiple techniques and phases

- Comprehensive processes, techniques, and phases
- *Dimension: Problem domain*
 - Single artifact
 - Single project
 - Single domain
 - Multiple domains
 - Comprehensive domain coverage
- *Dimension: Artifact scale*
 - Units/components
 - Subsystems
 - Small-scale systems
 - Large-scale systems
 - System-of-systems
- *Dimension: Subject expertise level*
 - Junior expertise
 - Intermediate expertise
 - Advanced expertise
- Category: Understanding
 - *Dimension: Evidence*
 - Research conjectures
- Supportive observations
- Common patterns/similarities
- Replicated results
- Validated theories
- Category: Impact
 - *Dimension: Science/engineering*
 - Subset of Empirical Software Engineering
 - Empirical Software Engineering
 - Software Engineering
 - Computer Science
 - Systems Engineering and/or Information Technology
 - *Dimension: Industry*
 - Isolated examples
 - Organizational adoption
 - Multi-organizational adoption
 - Industry-wide shift